

The AI Legislative Analyzer: Optimizing Token Compression and Information Density for Sustainable Legal Summarization

Introduction to the Civic and Computational Challenge

The Indian legal system represents one of the most extensive and linguistically complex civic frameworks globally. Legislative proposals introduced in the Lok Sabha or Rajya Sabha, subsequent gazette notifications, and judicial rulings are characterized by archaic terminology, protracted clause structures, and nested conditional logic. These documents frequently span hundreds of pages, rendering them largely impenetrable to the average citizen. While the fundamental principles of a participatory democracy demand that civic policies be accessible and comprehensible, traditional manual summarization is wholly outpaced by the sheer volume and velocity of the nation's legislative output.

The advent of Large Language Models (LLMs) presents a transformative mechanism for automated, abstractive summarization. By deploying a centralized "Citizen's Dashboard," it is theoretically possible to translate verbose parliamentary bills into simplified, real-time civic intelligence. However, the operationalization of this concept encounters a severe computational bottleneck: Indian legal documents routinely exceed the 100,000-token threshold. Processing contexts of this magnitude utilizing frontier transformer architectures introduces exponential computational overhead.

Furthermore, the continuous ingestion and summarization of massive legal corpora exacts an unsustainable environmental toll. The energy consumed during LLM inference—particularly for extended contexts—has escalated into a critical global sustainability crisis. Recent lifecycle assessments indicate that the cumulative energy consumption and carbon footprint of inference now rival or exceed the emissions generated during the initial training of these massive models. Consequently, deploying a continuous legislative analyzer requires reconciling the demand for high-fidelity semantic extraction with strict ecological and infrastructural constraints.

The architectural solution necessitates a paradigm shift toward maximizing "Information Density"—defined as the semantic and civic value delivered per computational token consumed. By integrating advanced token compression techniques, the system can algorithmically identify and excise semantic redundancy, shrinking raw, 100,000-token legal

documents into highly concentrated, high-density prompts. This forced compression directly truncates the sequence length fed into the LLM, neutralizing the quadratic scaling of the attention mechanism and directly reducing the kilowatt-hour expenditure and carbon footprint of the inference phase. This report exhaustively details the architectural, mathematical, and infrastructural framework required to build an AI Legislative Analyzer that operates at the intersection of deep natural language processing, Green AI sustainability, and civic legal design.

The Topography of Indian Legal Documentation

To engineer a pipeline capable of real-time summarization, the inherent structure and linguistic topography of the source material must be rigorously analyzed. Indian legislative proposals are not monolithic blocks of text; they are highly structured documents bound by strict parliamentary conventions. A standard Bill encompasses a Long Title, a Short Title, an Enacting Formula, and an extensive arrangement of Clauses, Sub-clauses, and Schedules. Furthermore, Bills are accompanied by a Statement of Objects and Reasons, a memorandum regarding delegated legislation, and a Financial Memorandum detailing expenditures from the Consolidated Fund of India. Once enacted and assented to by the President, these statutes, along with countless subordinate rules and ordinances, are published in the Gazette of India.

The primary obstacle for natural language processing in this domain is the distinct syntactic behavior of Indian legal English. The domain exhibits an absence of fixed word order, frequent use of passive voice, and a highly specialized, jurisdiction-specific lexicon. Empirical evaluations demonstrate that domain-independent, state-of-the-art models struggle to parse these documents effectively without severe hallucination, context dilution, or outright omission of critical legal stipulations. Furthermore, the historical scarcity of extensive, publicly available, jurisdiction-specific training datasets for Indian legal summarization has impeded the fine-tuning of abstractive models.

To counteract these domain-specific vulnerabilities, the architectural framework must incorporate a robust text normalization and entity extraction phase prior to any generative summarization. This relies on the deployment of specialized, pre-trained encoders such as Legal-BERT or InLegalBERT. Research utilizing the NER-IPL (Indian Legal Prediction Dataset for Named Entity Recognition), a comprehensive dataset comprising over 213,000 sentences and 6.19 million tokens, demonstrates that models like InLegalBERT achieve superior F1 scores in isolating critical legal entities. By executing Legal Named Entity Recognition (L-NER) to identify specific statutes, penal codes, jurisdictions, and monetary figures, the system establishes a protective semantic layer.

Additionally, the corpus of Indian judicial and legislative texts requires the identification of specific rhetorical roles. Sentences must be segmented into categories such as arguments, precedents, statutory reliance, and final rulings. Frameworks trained on specialized corpora, such as the Indian Legal Documents Corpus (ILDC) and the Multilingual Indian Legal Document Summarization (MILDSum) dataset, enable the pre-processing layer to map the structural

skeleton of the 100,000-token document. By explicitly defining these rhetorical roles and freezing the identified named entities, the system guarantees that the aggressive token pruning executed in subsequent stages does not inadvertently strip the document of its core legal facts and enforceable clauses.

The Ecological Imperative: Green AI and Inference Metrology

The deployment of an LLM-driven public dashboard operating on a national scale generates millions of automated queries and background summarization tasks daily. While an isolated prompt to a frontier model may consume between 0.3 and 1.0 watt-hours (Wh) of electricity, scaling this operational cadence to encompass the continuous, real-time tracking of India's legislative apparatus results in gigawatt-hours of energy consumption. The environmental impact of Large Language Models is rising significantly, with inference now accounting for more than half of their total lifecycle carbon emissions.

Quantifying the Carbon Footprint of Inference

The total carbon footprint of LLM infrastructure is partitioned into operational emissions, stemming from energy consumed during runtime, and embodied emissions, arising from hardware manufacturing, infrastructure construction, and hardware depreciation. In modern AI data centers, operational emissions constitute up to 70% of the total inference-related carbon output.

Recent comprehensive life cycle assessments (LCAs) underscore the severity of this shift. For instance, an analysis of the Mistral Large 2 model revealed that while the 18-month training phase generated 20.4 kilotons of carbon dioxide equivalent (CO_2eq), a single average inference query comprising merely 400 tokens caused 1.14 gCO_2e and consumed 45 milliliters of fresh water for data center cooling. When extrapolated to a platform processing documents exceeding 100,000 tokens across thousands of concurrent sessions, the environmental degradation scales exponentially. Highly complex symbolic and abstract domains, such as legal interpretation, consistently demand more computation and yield lower energy efficiency, exacerbating the ecological footprint.

To construct a sustainable Legislative Analyzer, continuous, granular metrology is required. The architecture integrates Green AI frameworks, such as EcoLogits and CodeCarbon, to dynamically estimate the environmental impacts of LLM inference through API telemetry. The underlying mathematical formulation utilized to calculate the operational carbon footprint (CF) during the inference process is defined as:

$$CF = E \times \alpha$$

Where E denotes the total energy consumed in kilowatt-hours (kWh), and α represents the carbon emission factor of the specific electricity grid (in $kgCO_2/kWh$) where the inference is executed. The total energy E is deeply entangled with the Power Usage Effectiveness (PUE) of the hosting data center, factoring in the cooling and auxiliary infrastructure overhead.

The Phase-Level Energy Dynamics of Transformers

A critical insight driving the necessity of token compression lies in the phase-level analysis of LLM inference energy consumption. The generation of text by a transformer model is divided into two distinct phases: the "prefill" phase and the "decoding" phase.

During the prefill phase, the model ingests the entirety of the input prompt, computing the key-value (KV) cache and building internal semantic representations. Because the self-attention mechanism scales quadratically with the sequence length, processing a 100,000-token parliamentary bill demands an astronomical number of floating-point operations (FLOPs). Subsequently, during the decoding phase, the model generates output tokens autoregressively. Empirical benchmarking reveals a critical dependency: increases in the prefill cost disproportionately amplify the energy cost per token during the decoding phase, with amplifications ranging from 1.3% to 51.8% depending on the model's parameter count and architecture.

Therefore, redundant or verbose prompts not only inflate the immediate energy required to read the document but permanently degrade the energy efficiency of the model while it generates the summary. Limiting the length of the input prompt via algorithmic compression is the single most mathematically effective lever available to systems engineers for minimizing E and stabilizing the Time to First Token (TTFT) latency metric.

Green AI Metric	Operational Definition	Impact of 100k+ Context Constraints	Primary Systemic Mitigation Strategy
Energy per Query (Wh)	Electricity required to process a prompt and compute the KV cache.	Scales quadratically; massively degrades efficiency for long documents.	Aggressive token compression to forcibly reduce input sequence length.

Carbon Footprint (gCO_2e_l)	Greenhouse gas emissions tied to regional grid carbon intensity.	Highly elevated in regions utilizing fossil-fuel energy grids.	Compute routing to green data centers; dynamic prompt pruning.
Time to First Token (TTFT)	Latency reflecting the computational duration of the prefill phase.	Severe lag; degrades the real-time responsiveness of the dashboard.	Segmented context compression and token-level perplexity filtering.
Water Consumption (WCF)	Milliliters of fresh water evaporated for data center thermal management.	Increases linearly with prolonged GPU utilization times.	Maximizing output Information Density to minimize generation duration.

The Mechanics of Token Compression: The LLMLingua Paradigm

To ingest expansive legal corpora into a reasoning LLM while strictly adhering to Green AI parameters, the pipeline relies on the architectural intermediary of prompt compression. Traditional Retrieval-Augmented Generation (RAG) models attempt to solve the context limit by retrieving only the most relevant document chunks via vector similarity. However, chunking a densely interconnected legal document frequently destroys the structural coherence of the legislation. Passing thousands of retrieved chunks introduces immense noise, dilutes the model's focus, and invariably triggers the "lost in the middle" phenomenon—a documented flaw where LLMs fail to retrieve or reason over information buried in the center of extended context windows.

State-of-the-art token compression frameworks, most notably the LLMLingua series, address this by functioning as a distillation layer. LLMLingua operates on the information-theoretic principle of "LM is Compression" and exploits the inherent statistical redundancy of natural language.

Perplexity-Based Pruning and Entropy

The core engine of LLMLingua utilizes a highly efficient, small-parameter language model (such as LLaMA-7B, Qwen, or GPT-2-small) to evaluate the perplexity (or surprisal) of each token within the raw legislative text. In information theory, tokens with low perplexity are highly predictable given their surrounding context; they carry minimal entropy and provide negligible

semantic utility to a highly capable generative model.

By establishing a perplexity threshold, LLMLingua systematically strips these highly predictable tokens—including conjunctions, redundant legal boilerplate, and filler syntax—creating a compressed token sequence. While this heavily pruned output appears ungrammatical, disjointed, and largely illegible to human readers, the target frontier LLM retains the capacity to decode the latent semantic structure and reason over it perfectly.

The LLMLingua architecture incorporates three synergistic components to achieve this without corrupting the meaning:

1. **Budget Controller:** This module dynamically assigns varying compression ratios to different segments of the prompt. For instance, instructional prompts and demonstration examples receive lower compression rates to maintain task alignment, while the raw legislative text is subjected to aggressive pruning.
2. **Iterative Token-Level Compression:** Rather than compressing the document in a single pass, the algorithm divides intermediate results into segments and refines the conditional dependencies between tokens iteratively. The perplexity of a token is continuously re-evaluated based on the preceding compressed segments, ensuring that critical syntactic bridges are preserved.
3. **Distribution Alignment:** To ensure the small compression model accurately mimics the linguistic expectations of the larger, black-box generating model (e.g., GPT-4 or Claude 3.5), the small model is fine-tuned using instruction data generated by the target LLM. This aligns the semantic distributions between the compressor and the generator.

Empirical evaluations confirm that LLMLingua can achieve up to a 20x compression ratio while preserving the original reasoning and summarization capabilities of the prompt. This translates to an immediate, profound reduction in API costs, memory allocation, and KV cache bloat.

Scaling to Massive Contexts: The LongLLMLingua Architecture

While the baseline LLMLingua excels at general prompt compression, scaling to legislative bills and supreme court judgments that exceed 100,000 tokens requires specialized adaptations. Standard perplexity pruning over such vast horizons risks indiscriminately deleting vital context, particularly in dense legal arguments where every clause modifies the preceding statute. LongLLMLingua builds upon the baseline framework to specifically tackle extended contexts by transforming the compression process into a *question-aware* and *position-aware* mechanism.

Question-Aware Contrastive Perplexity

In a civic dashboard environment, the system is often tasked with generating targeted summaries, such as isolating the financial impact of a bill on medium enterprises. LongLLMLingua calculates the contrastive perplexity of the document's tokens conditioned

specifically on this query. It compares the perplexity distribution of the document alone against the perplexity distribution of the document concatenated with the target question. Tokens that exhibit a significant shift in perplexity when the question is introduced are mathematically identified as highly relevant to the civic query and are shielded from deletion.

The system executes a coarse-to-fine compression hierarchy. Initially, it discards entirely irrelevant paragraphs or schedules (coarse-grained). Subsequently, it follows the iterative token-level compression algorithm but dynamically assigns the compression budget τ_i to each retained document chunk x_{doc}^k according to the ranking index $I(r_k)$ of its importance score from the coarse-grained compression. The dynamic budget allocation is formulated as:

$$\tau_i = \tau_{doc}^k, \forall x_i \in x_{doc}^k$$

$$\tau_{doc}^k = \max \left(\min \left(\left(1 - \frac{2I(r_k)}{K} \right) \delta_\tau + \tau_{doc}, 1 \right), 0 \right)$$

Where i and k are the indices of the token and document respectively, K denotes the total number of documents, and δ_τ is a hyper-parameter controlling the variance of the dynamic allocation.

Mitigating Position Bias and Ensuring Entity Integrity

To counteract the "lost in the middle" position bias inherent in transformer attention heads, LongLLMLingua introduces a document reordering mechanism. It intentionally shifts the most vital, high-entropy document chunks to the extreme ends of the compressed prompt (the absolute beginning and the absolute end), regions where modern LLMs exhibit near-perfect information recall.

Furthermore, legal summarization cannot tolerate the hallucination or corruption of named entities. Compressing entities at the sub-word token level often leads to critical errors in the generated output (e.g., misstating a fine of "10,000 Rupees" as "1,000 Rupees"). To secure absolute accuracy, LongLLMLingua utilizes a post-compression subsequence recovery strategy. By iterating through the tokens in the generated response and executing a longest-substring matching algorithm against the original uncompressed text, the system forcefully restores the exact original phrasing of critical legal entities, ensuring statutory fidelity.

Benchmarking on complex, multi-document reasoning tasks demonstrates that LongLLMLingua yields a performance boost of up to 21.4% over uncompressed prompts while utilizing merely 25% of the original token count. Concurrently, it accelerates the end-to-end system latency by a factor of 1.4x to 3.8x, yielding a 94% reduction in computational costs. By

embedding LongLLMLingua within the Legislative Analyzer, the architecture circumvents the 100,000-token barrier while fundamentally minimizing the kilowatt-hour burn rate of the prefill phase.

Maximizing Information Density and Value Per Token (VPT)

The ultimate metric for judging the efficacy of the AI Legislative Analyzer is "Information Density." Within the paradigm of Green AI and sustainable software development, efficiency is not merely a measurement of inference speed; it is an economic and semantic ratio defining the utility delivered per computational unit expended.

The Value Per Token (VPT) Paradigm

In traditional software operations, success is measured via Total Cost of Ownership (TCO). In the generative AI landscape, this metric evolves into Value Per Token (VPT). VPT is formulated mathematically as the tangible business or civic value delivered divided by the total tokens consumed during both prefill and generation.

If an LLM generates a highly verbose, 2,000-word summary of a legislative bill that contains the exact same factual payload as a tightly woven 300-word summary, the VPT of the former is abysmally low. The excess token generation is classified not merely as poor writing, but as literal ecological and computational waste. Every unnecessary token generated draws unnecessary GPU power, contributing to the data center's carbon footprint.

Mathematical Formulation of Entity-Token Density

To optimize VPT, the dashboard must force the generative model to output summaries that are intensely concise yet saturated with meaning. This requires moving beyond subjective assessments of "conciseness" and utilizing deterministic, quantitative metrics. The primary metric utilized is Entity-Token Density (ETD).

Utilizing standard Natural Language Processing pipelines equipped with libraries such as spaCy (specifically the `en_core_web_sm` model), the ETD of a generated summary is calculated by isolating the number of distinct named entities (excluding basic cardinal numbers to avoid noise) and dividing it by the total token volume:

$$\text{Entity-Token Density (ETD)} = \frac{\text{Number of Entities}}{\text{Number of Tokens}}$$

A low ETD indicates a summary plagued by verbosity and sparse factual grounding. A high ETD indicates a tightly compressed, highly informative text.

Chain of Density (CoD) and Dialectical Summarization

Achieving optimal Entity-Token Density requires deliberate, structured prompting strategies to override the natural tendency of LLMs to generate verbose, conversational text. The "Chain of Density" (CoD) framework is an iterative methodology designed to systematically enrich summaries while strictly constraining their length.

Rather than generating a summary in a single, zero-shot pass, the CoD prompt forces the LLM to produce an initial, sparse summary. Over multiple internal iterations (Internal Step Responses), the model is instructed to identify 1 to 3 highly salient, missing entities from the source document and explicitly fuse them into the existing text. Crucially, the model is commanded to maintain an identical word count across every iteration.

To accommodate the newly introduced legal entities without expanding the token budget, the model is forced to aggressively abstract, compress, and rephrase its own earlier generations. This constraint acts as a hydraulic press, forcing the output away from verbose, extractive copying toward a highly synthesized, abstractive narrative. Human preference studies indicate that readers strongly prefer these densely packed summaries, provided the entity density remains balanced—optimally around 0.15 entities per token. Exceeding this threshold results in a dense "entity soup" that compromises grammatical readability.

An advanced extension of this concept integrated into the architecture is the "Chain of Summaries" (CoS), which utilizes a structure inspired by Hegel's dialectical method. This approach generates an initial summary (the thesis), critiques the summary by generating questions about missing legislative context or ambiguous clauses (the antithesis), and finally produces an upgraded summary that resolves these gaps (the synthesis).

By combining LongLLMLingua for absolute *input* compression and Chain of Density for rigorous *output* generation, the AI Legislative Analyzer ensures minimal energy consumption during the prefill phase and maximum semantic payload during the decoding phase, achieving peak Value Per Token.

System Architecture: The Real-Time Civic Dashboard

To operationalize these advanced theoretical models, the AI Legislative Analyzer requires a resilient, highly automated data pipeline capable of bridging raw, unstructured government repositories with the citizen-facing interface.

Data Ingestion and Pipeline Orchestration

The foundation of the architecture relies on high-velocity, real-time data ingestion. Indian legal updates, gazette notifications, and parliamentary bills are distributed asynchronously across disparate platforms, including the Open Government Data Platform (data.gov.in), API Setu, PRS Legislative Research, and the official eGazette portal (egazette.gov.in).

The ingestion layer utilizes automated, proxy-rotated web scraping agents and API connectors to monitor these portals. Because government websites frequently employ rate-limiting or anti-bot measures (such as 403 Forbidden errors or CAPTCHAs), the scraper relies on residential IP rotation to ensure continuous uptime without triggering security blockades. Furthermore, to avoid redundant processing and unnecessary compute expenditure, the pipeline integrates Change Data Capture (CDC) software. CDC continuously mines the metadata of the government servers, guaranteeing that the pipeline only ingests and processes newly published or formally amended documents.

Because a vast majority of Indian gazettes are published as complex, digitally signed PDF documents, the ingestion layer routes the files through an advanced Optical Character Recognition (OCR) and layout-parsing module. This module identifies and isolates Tables of Contents, normalizes whitespace, and extracts the text while rigorously preserving the hierarchical layout of Sections, Clauses, and Schedules. The cleaned, unstructured text is then passed into a high-throughput event streaming platform, such as Apache Kafka, which queues the documents for asynchronous processing, preventing systemic bottlenecks during periods of high legislative activity.

Legal-Aware Preprocessing and Compression Routing

Once queued, the text undergoes the critical legal-aware preprocessing phase. The custom Legal-BERT encoder tags rhetorical roles, isolates the core arguments, and performs L-NER to freeze statutory entities.

If the document exceeds the defined token threshold (e.g., 10,000 tokens), it is dynamically routed to the LongLLMLingua compression module. To adhere to strict Green AI parameters, this compression is executed by a locally hosted, highly quantized small language model (such as a 4-bit quantized LLaMA-3-8B). Performing the perplexity calculations locally eliminates the latency and monetary cost of round-trip API calls to commercial providers. The budget controller is automatically calibrated to apply an optimal compression ratio—frequently up to 80%—effectively shrinking a massive 100,000-token bill into a dense, 20,000-token prompt heavily concentrated with high-entropy legal concepts and immutable named entities.

Generative Summarization and Plain Language Translation

The hyper-compressed prompt, fused with the Chain of Density instruction template, is subsequently dispatched to a frontier reasoning LLM (such as GPT-4o or Claude 3.5) for generation.

However, high information density must not equate to civic incomprehensibility. If the output remains steeped in legalese, it fails the primary objective of the Citizen's Dashboard. Therefore, the final iteration of the CoD prompt enforces strict "Plain Language" translation parameters, heavily influenced by initiatives like the Vidhi Centre for Legal Policy's SARAL (Simple, Accessible, Rational and Actionable Laws) framework.

The dashboard's presentation layer decodes the complex policy into highly accessible, structured visualizations rather than monolithic paragraphs. The final output delivered to the citizen includes:

- **Executive Overviews:** 100-word, high-density abstracts conveying the bill's primary intent.
- **Targeted Impact Assessments:** Segmented, bulleted implications outlining exactly how the legislation affects specific civic demographics (e.g., local MSMEs, taxpayers, agricultural workers).
- **Momentum and Trajectory Trackers:** Visualized progress pathways tracking the bill's movement through standing committees, the Lok Sabha, and the Rajya Sabha.

Architectural Layer	Core Technologies & Techniques	Primary Function within the Pipeline	Direct Sustainability & Efficiency Benefit
Ingestion & Streaming	Apache Kafka, CDC, Proxy-Scraping, Structure-Aware OCR	Real-time tracking of eGazette and Parliamentary feeds.	Eliminates redundant processing of unmodified or duplicate legal documents.
Normalization	Legal-BERT, L-NER, Rhetorical Role Tagging	Identification of clauses, precedents, and entities.	Ensures that subsequent aggressive compression does not destroy vital legal facts.
Input Compression	LongLLMLingua (Locally hosted quantized SLM)	Contrastive perplexity-based pruning of 100k+ contexts.	Drastically truncates prefill sequence lengths, slashing Wh consumption and latency.
Generation & Translation	Frontier LLM + Chain of Density (CoD) Prompting	Iterative drafting to maximize Entity-Token Density.	Minimizes output token generation, reducing decoding compute and optimizing VPT.

Evaluation Metrics for Legal Summarization

Validating the outputs of the AI Legislative Analyzer requires moving beyond legacy evaluation frameworks. Traditional n-gram overlap metrics, such as BLEU or standard ROUGE, are fundamentally insufficient for evaluating abstractive summarization in the legal domain. These metrics penalize the model for executing valid semantic paraphrasing—the exact mechanism required to translate archaic Indian law into plain language.

While ROUGE-L is maintained purely to measure structural recall, the system relies heavily on BERTScore. BERTScore utilizes contextualized token embeddings to compute similarity accurately, avoiding penalties for semantic rearrangements and seamlessly capturing long-range dependencies. This ensures that the fundamental meaning of the law remains uncorrupted despite the aggressive 80% input compression and the plain-language translation.

Furthermore, general-purpose LLM benchmarks are inadequate for legal evaluation. The system's performance is continuously validated against specialized, jurisdiction-aware benchmarks such as LexSumm and InLegalSum. These datasets are specifically curated to test summarization capabilities across varying legal jurisdictions, forcing the model to demonstrate competence in identifying issue-spotting, rule-recall, and rhetorical functions unique to the Indian legal system.

Uncertainty Quantification and Hallucination Mitigation

The highest risk associated with utilizing LLMs for legal interpretation is the generation of hallucinations—fabricating statutes, citing non-existent precedents, or misrepresenting financial clauses. To deploy the dashboard securely, the pipeline integrates rigorous Uncertainty Quantification (UQ).

Advanced UQ methods shift away from simple confidence scores and utilize density-based methods to evaluate the truthfulness of the generated text. By extracting token embeddings from the middle layers of the generative LLM, the system computes the Mahalanobis Distance (MD) for each generated sequence against a baseline empirical covariance matrix.

$$U_{RMD}(x, l) = U_{MD}(x, l) - U_{MD_0}(x, l)$$

This relative Mahalanobis Distance provides a highly robust, mathematically sound uncertainty score. If the MD score exceeds a specific threshold, the generated summary is flagged for hallucination risk and routed for either automatic regeneration with a lower temperature setting or human-in-the-loop review. This guarantees the highest level of factual integrity before the data is published to the public dashboard.

Conclusion

The architecture of the AI Legislative Analyzer bridges a critical, expanding chasm between democratic accessibility and computational sustainability. Indian legal documents and parliamentary bills possess a morphological density and structural complexity that intrinsically resist straightforward interpretation by the citizenry. Yet, scaling traditional, unoptimized LLM inference to automatically summarize these massive texts presents an insurmountable ecological and financial burden, driven by the exponential energy costs associated with processing 100,000+ token contexts.

By subordinating the entire engineering pipeline to the optimization of Information Density and Value Per Token, this framework resolves the paradox. The deployment of localized, perplexity-evaluating small language models via the LongLLMLingua architecture algorithmically identifies and excises semantic redundancy, compressing vast legislative texts into highly concentrated prompts without sacrificing legal fidelity. Concurrently, generative frameworks driven by Chain of Density prompting guarantee that the resultant plain-language output delivers maximum civic utility per generated token.

This dual-layered optimization—radically compressing the input while densifying the output—dramatically curtails the kilowatt-hour expenditure, cooling water requirements, and subsequent carbon footprint of the system. Ultimately, the integration of advanced, context-aware token compression transforms the concept of real-time, citizen-facing legal analysis from a computationally prohibitive theory into a scalable, ecologically responsible reality, ensuring that the laws governing society are accessible to all without accelerating the degradation of the environment.